ELECTRONIC ASSIGNMENT COVERSHEET

Murdoch UNIVERSITY

Student Number	35052544
Surname	Doheon
Given name	Kim
Email	dylankim1306@gmail.com

Unit Code	ICT283
Unit name	data structures and abstractions
Enrolment mode	Face to face
Date	17 April, 2026
Assignment number	1
Assignment name	assignment 1
Tutor	Shri Rai

Student's Declaration:

- Except where indicated, the work I am submitting in this assignment is my own work and has not been submitted for assessment in another unit.
- This submission complies with Murdoch University's academic integrity commitments. I am aware that information about plagiarism and associated penalties can be found at http://www.murdoch.edu.au/teach/plagiarism/. If I have any doubts or queries about this, I am further aware that I can contact my Unit Coordinator prior to submitting the assignment.
- I acknowledge that the assessor of this assignment may, for the purpose of assessing this assignment:
  - reproduce this assignment and provide a copy to another academic staff member; and/or
  - submit a copy of this assignment to a plagiarism-checking service. This web-based service may retain a copy of this work for the sole purpose of subsequent plagiarism checking, but has a legal agreement with the University that it will not share or reproduce it in any form.
- I have retained a copy of this assignment.
- I will retain a copy of the notification of receipt of this assignment. If you have not received a receipt within three days, please check with your Unit Coordinator.

I am aware that I am making this declaration by submitting this document electronically and by using my Murdoch ID and password it is deemed equivalent to executing this declaration with my written signature.

Optional Comments to Tutor: E.g. If this is a group assignment, list group members here

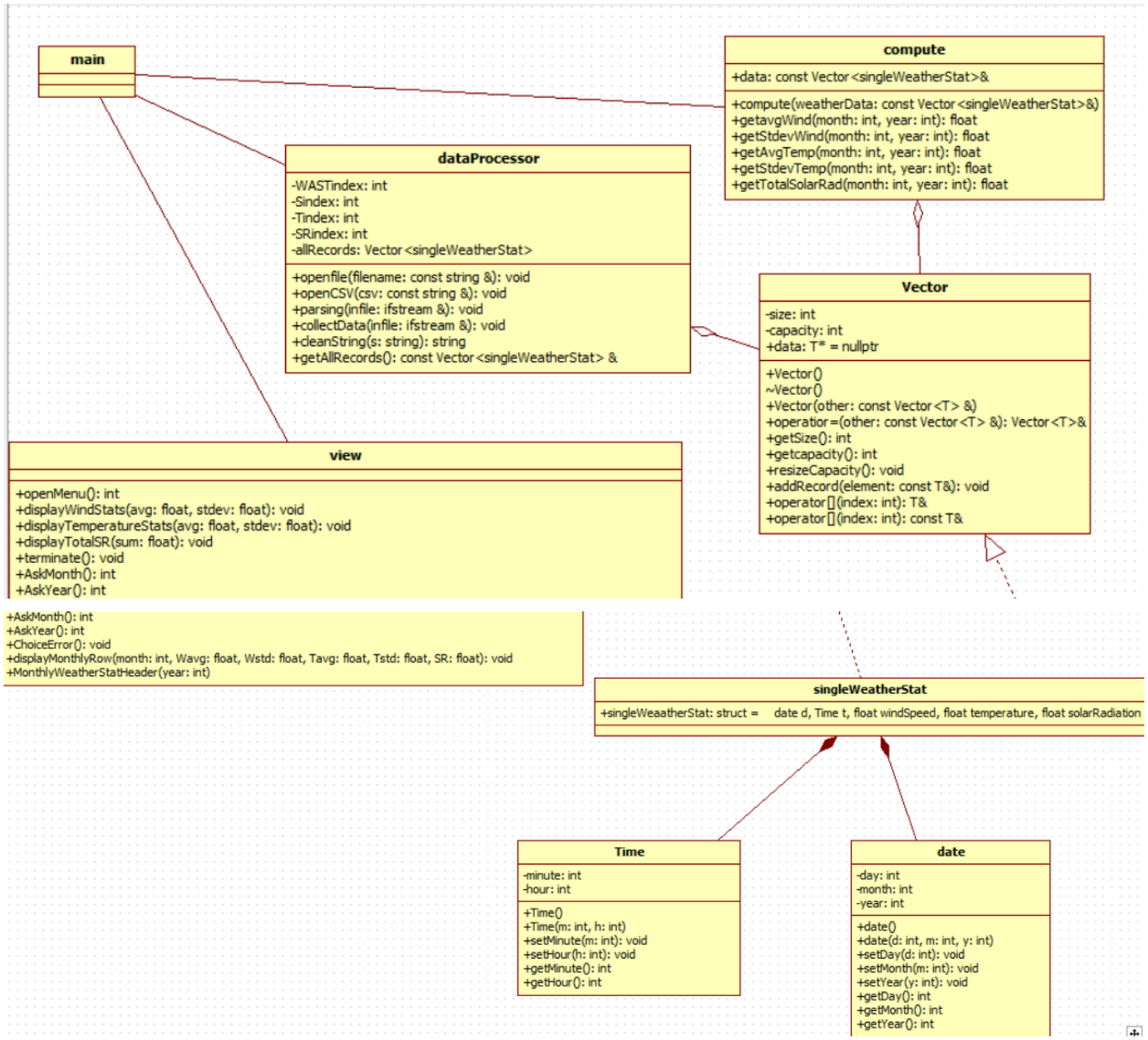
*If you can, please insert this completed form into the body of **each** assignment you submit. Follow the instructions in the Unit Information and Learning Guide about how to submit your file(s) and how to name them, so the Unit Coordinator knows whose work it is.*

ICT283 Assignment1

Doheon Kim 35052544

UML design and data dictionary

A screenshot of UML diagram



To ensure a clean MVC architecture, I designed the View class to focus only on user interaction—handling console output and passing user input back to the Main function. In this design, Main acts as the Controller, sequentially calling methods from both the View and various Models to prevent them from interacting directly.

While the project requirements might have allowed the data processing logic to reside within Main, I chose to keep them separated by having a DataProcessor class. My goal was to keep the Main doing one job (as a controller). The DataProcessor is responsible for reading CSV data from text files and extracting some data and storing them in a custom template Vector class.

For the mathematical calculation, I implemented a Compute class. This component handles different sorts of calculations such as average wind speed and standard deviation. Throughout the development, I tried to enhance memory efficiency by utilizing const references as much as possible. For instance, the Compute class accesses the data stored by the DataProcessor via a const reference rather than by value. This approach allows the class to process information directly without copying large vector objects, while simultaneously ensuring data integrity by preventing accidental modifications.

Final thing to note is that the template Vector class was designed to mimic the behavior of a standard dynamic array. It supports automatic resizing and dynamic element storage for any data type. In this specific implementation, the vector stores instances of a singleWeatherStat (struct) as its primary data elements.

Data dictionary

Entity	Attribute	Data Type	Description
date	day	int	Day of the month
	month	int	Month of the year
	year	int	Four-digit year
Time	minute	int	Minutes past the hour
	hour	int	Hour of the day

singleWeatherStat	d	date	Composition: Stores the date of the record.
	t	Time	Composition: Stores the time of the record.
	windSpeed	float	Speed of wind
	temperature	float	Temperature
	solarRadiation	float	Solar intensity
Vector	data	T* (Pointer)	Dynamic array holding elements of type `T`.
	size	int	Number of elements currently stored.
	capacity	int	Total allocated memory slots.
dataProcessor	allRecords	Vector	The primary storage for every weather entry.

	WASTindex	int	CSV column index for Date/Time string.
	Sindex	int	CSV column index for Wind Speed.
	Tindex	int	CSV column index for Temperature.
	SRindex	int	CSV column index for Solar Radiation.
compute	data	const Vector&	Alias/Reference to the **allRecords** vector.

ALGORITHM

```
FUNCTION Main()
    DECLARE myDP AS dataProcessor
    DECLARE hardCodedFile AS STRING <- "data/data_source.txt"

    myDP.openfile(hardCodedFile)
    DECLARE allRecords AS Vector<singleWeatherStat> = myDP.getAllRecords()

    DECLARE MyView AS view
```

```
DECLARE calculator AS compute(allRecords)

DECLARE loopFlag AS BOOLEAN <- TRUE
WHILE (loopFlag) DO
   DECLARE choice AS INTEGER
      choice <- MyView.openMenu()

   SWITCH (choice) DO
      CASE 1:
         DECLARE m AS INTEGER <- MyView.AskMonth()
         DECLARE y AS INTEGER <- MyView.AskYear()
         DECLARE avg AS FLOAT <- calculator.getAvgWind(m, y)
         DECLARE stdev AS FLOAT <- calculator.getStdevWind(m, y)
         MyView.displayWindstats(avg, stdev)

      CASE 2:
         DECLARE m AS INTEGER <- MyView.AskMonth()
         DECLARE y AS INTEGER <- MyView.AskYear()
         DECLARE avg AS FLOAT <- calculator.getAvgTemp(m, y)
         DECLARE stdev AS FLOAT <- calculator.getStdevTemp(m, y)
         MyView.displayTemperatureStats(avg, stdev)

      CASE 3:
         DECLARE m AS INTEGER <- MyView.AskMonth()
         DECLARE y AS INTEGER <- MyView.AskYear()
         DECLARE sum AS FLOAT <- calculator.getTotalSolarRad(m, y)
         MyView.displayTotalSR(sum)

      CASE 4:
         DECLARE y AS INTEGER <- MyView.AskYear()
         MyView.MonthlyWeatherStatsHeader(y)
         FOR m FROM 0 TO 12 DO
            DECLARE Wavg AS FLOAT <- calculator.getAvgWind(m, y)
            DECLARE Wstdev AS FLOAT <- calculator.getStdevWind(m, y)
            DECLARE Tavg AS FLOAT <- calculator.getAvgTemp(m, y)
            DECLARE Tstdev AS FLOAT <- calculator.getStdevTemp(m, y)
            DECLARE SRsum AS FLOAT <- calculator.getTotalSolarRad(m, y)

            MyView.displayMonthlyRow(m, Wavg, Wstdev, Tavg, Tstdev, SRsum)

         END FOR

      CASE 5:
         loopFlag <- FALSE
```

```
            MyView.terminate()

        DEFAULT:
            MyView.ChoiceError()
    END SWITCH
  END WHILE

  RETURN 0
END FUNCTION




CLASS view

  FUNCTION openMenu() RETURNS INTEGER
    DECLARE choice AS INTEGER

    WRITE"option1: select year and month to get average speed and standard deviation for
the wind"
    WRITE"option2: select year and month to get average and standard deviation of
temperature"
    WRITE "option3: solar radiation in KWh/m*m"
    WRITE "option4: select a year to print all data above in each month"
    WRITE "option5: terminate the program"

    WRITE "please select option:"
    READ KB, choice

    RETURN choice
  END FUNCTION




  FUNCTION displayWindstats(avg AS FLOAT, stdev AS FLOAT)
    WRITE"average speed and standard deviation for the wind: ", avg, " and ", stdev
  END FUNCTION




  FUNCTION displayTemperatureStats(avg AS FLOAT, stdev AS FLOAT)
```

```
    WRITE"average and standard deviation for the temperature: ", avg, " and ", stdev
  END FUNCTION



  FUNCTION displayTotalSR(sum AS FLOAT)
    WRITE "the total of solar radiation in kmh/(m*m) is: ", sum
  END FUNCTION



  FUNCTION terminate()
    WRITE "ending the program.. press any key to exit"
  END FUNCTION



  FUNCTION AskMonth() RETURNS INTEGER
    DECLARE M AS INTEGER
    WRITE "enter a month: "
    READ KB, M
    RETURN M
  END FUNCTION



  FUNCTION AskYear() RETURNS INTEGER
    DECLARE Y AS INTEGER
    WRITE "enter a year: "
    READ KB, Y
    RETURN Y
  END FUNCTION



  FUNCTION ChoiceError()
    WRITE "you need to select between 1 and 5!"
  END FUNCTION

  FUNCTION MonthlyWeatherStatsHeader(year AS INTEGER)
    WRITE "annual summary for ", year
    WRITE "month: | average wind speed(standard deviation), average temperature(standard
deviation), total solar radiation"
```

```
    END FUNCTION



    FUNCTION displayMonthlyRow(month AS INTEGER, Wavg AS FLOAT, Wstd AS FLOAT,
Tavg AS FLOAT, Tstd AS FLOAT, SR AS FLOAT)
        WRITE month, ":"
        WRITE Wavg, Wstd, Tavg, Tstd, SR
    END FUNCTION

END CLASS VIEW




CLASS dataProcessor

    FUNCTION openfile(filename AS STRING)
            OPEN READ openTxt= “filename”
        IF NOT (openTxt) THEN
            WRITE "failed to open file"
            RETURN
        END IF

        DECLARE csv AS STRING

        WHILE (getline(openTxt, csv)) DO
            openCSV("data/" + csv)
        END WHILE
        openTxt.close()
    END FUNCTION



    FUNCTION openCSV(csv AS STRING)
            OPEN READ infile =”csv”
        IF NOT (infile) THEN
            WRITE "failed to open file", csv
            RETURN
        END IF
```

```
    parsing(infile)
    infile.close()
END FUNCTION



FUNCTION parsing(infile AS ifstream)
    DECLARE headers AS STRING
    DECLARE column AS STRING
    DECLARE index AS INTEGER <- 0

    getline(infile, headers)
    DECLARE ss AS stringstream(headers)

    WHILE (getline(ss, column, ',')) DO
        DECLARE cleanedStr AS STRING
        cleanedStr <- cleanString(column)

        IF cleanedStr = "WAST" THEN
            WASTindex <- index
        ELSE IF cleanedStr = "S" THEN
            Sindex <- index
        ELSE IF cleanedStr = "T" THEN
            Tindex <- index
        ELSE IF cleanedStr = "SR" THEN
            SRindex <- index
        END IF
        index <- index + 1
    END WHILE
    collectData(infile)
END FUNCTION



FUNCTION cleanString(s AS STRING) RETURNS STRING
    DECLARE result AS STRING
    FOR EACH ( c IN s ) DO
        IF (c >= 'A' AND c <= 'z') THEN
            result <- result + c
        END IF
    END FOR

    RETURN result
END FUNCTION
```

```
FUNCTION collectData(infile AS ifstream)
   DECLARE line AS STRING
   DECLARE value AS STRING

   WHILE (getline(infile, line)) DO
      DECLARE ss AS stringstream(line)
      DECLARE index AS INTEGER <- 0
      DECLARE temporaryRecord AS singleWeatherStat

      WHILE getline(ss, value, ',') DO
         IF (index = WASTindex) THEN
            DECLARE ssWast AS stringstream(value)
            DECLARE datePart AS STRING
            DECLARE timePart AS STRING

            ssWast >> datePart >> timePart
            DECLARE d, m, y, hr, min AS INTEGER
            DECLARE separate AS CHAR

            stringstream(datePart) >> d >> separate >> m >> separate >> y
            stringstream(timePart) >> hr >> separate >> min

            temporaryRecord.d.setDay(d)
            temporaryRecord.d.setMonth(m)
            temporaryRecord.d.setYear(y)
            temporaryRecord.t.setHour(hr)
            temporaryRecord.t.setMinute(min)
         ELSE IF (index = Sindex) THEN
            stringstream(value) >> temporaryRecord.windSpeed
         ELSE IF (index = Tindex) THEN
            stringstream(value) >> temporaryRecord.temperature
         ELSE (IF index = SRindex) THEN
            stringstream(value) >> temporaryRecord.solarRadiation
         END IF
         index <- index + 1
      END WHILE

      allRecords.addRecord(temporaryRecord)
   END WHILE
END FUNCTION
```

```
  FUNCTION getAllRecords() RETURNS Vector<singleWeatherStat>
    RETURN allRecords
  END FUNCTION

END CLASS




CLASS compute

  FUNCTION getAvgWind(month AS INTEGER, year AS INTEGER) RETURNS FLOAT
    DECLARE sum AS FLOAT <- 0
    DECLARE count AS INTEGER <- 0
    FOR (i FROM 0 TO data.getSize() - 1) DO
      IF (data[i].d.getMonth() = month AND data[i].d.getYear() = year) THEN
        sum <- sum + data[i].windSpeed
        count <- count + 1
      END IF
    END FOR
    DECLARE avg AS FLOAT <- (3.6 * sum / count)
    RETURN avg
  END FUNCTION



  FUNCTION getStdevWind(month AS INTEGER, year AS INTEGER) RETURNS FLOAT
    DECLARE avg AS FLOAT <- getAvgWind(month, year)
    DECLARE varianceSum AS FLOAT <- 0
    DECLARE count AS INTEGER <- 0
    FOR (i FROM 0 TO data.getSize() - 1) DO
      IF (data[i].d.getMonth() = month AND data[i].d.getYear() = year ) THEN
        varianceSum <- varianceSum + (data[i].windSpeed - avg)
        count <- count + 1
      END IF
    END FOR
    RETURN sqrt(varianceSum / count)
  END FUNCTION
```

```
  FUNCTION getAvgTemp(month AS INTEGER, year AS INTEGER) RETURNS FLOAT
    DECLARE sum AS FLOAT <- 0
    DECLARE count AS INTEGER <- 0
    FOR (i FROM 0 TO data.getSize() - 1 ) DO
      IF (data[i].d.getMonth() = month AND data[i].d.getYear() = year ) THEN
        sum <- sum + data[i].temperature
        count <- count + 1
      END IF
    END FOR
    RETURN (sum / count)
  END FUNCTION



  FUNCTION getStdevTemp(month AS INTEGER, year AS INTEGER) RETURNS FLOAT
    DECLARE avg AS FLOAT <- getAvgTemp(month, year)
    DECLARE varianceSum AS FLOAT <- 0
    DECLARE count AS INTEGER <- 0
    FOR (i FROM 0 TO data.getSize() - 1 ) DO
      IF (data[i].d.getMonth() = month AND data[i].d.getYear() = year) THEN
        varianceSum <- varianceSum + (data[i].temperature - avg)
        count <- count + 1
      END IF
    END FOR
    RETURN sqrt(varianceSum / count)
  END FUNCTION



  FUNCTION getTotalSolarRad(month AS INTEGER, year AS INTEGER) RETURNS FLOAT
    DECLARE totalRadiation AS FLOAT <- 0
    FOR (i FROM 0 TO data.getSize() - 1) DO
      IF (data[i].d.getMonth() = month AND data[i].d.getYear() = year) THEN
        totalRadiation <- totalRadiation + data[i].solarRadiation
      END IF
    END FOR
    RETURN (totalRadiation / 6.0) / 1000.0
  END FUNCTION

END CLASS
```

```
TEMPLATE CLASS T
CLASS Vector

   FUNCTION Vector()
      DECLARE size AS INTEGER <- 0
      DECLARE capacity AS INTEGER <- 20
      DECLARE data AS NEW ARRAY OF T WITH capacity
   END FUNCTION



   FUNCTION ~Vector()
      DELETE data
         data <- NULLPTR
      size <- 0
      capacity <- 0
   END FUNCTION



   FUNCTION CopyConstructor(otherVector AS Vector<T>)

      DECLARE data AS NEW ARRAY OF T WITH otherVector.capacity
      size <- otherVector.size
      capacity <- otherVector.capacity

      FOR (i FROM 0 TO size - 1 ) DO
         data[i] <- otherVector.data[i]
      END FOR
   END FUNCTION



         FUNCTION operator=(other AS Vector<T>) RETURNS Vector<T>

      DECLARE newData AS NEW ARRAY OF T WITH other.capacity
```

```
        FOR ( i FROM 0 TO size - 1) DO
        newData[i] <- other.data[i]
    END FOR

        DELETE data
        data <- newData
    size <- other.size
    capacity <- other.capacity
        newData <- NULLPTR
    RETURN this
END FUNCTION



FUNCTION getSize() RETURNS INTEGER
    RETURN size
END FUNCTION



FUNCTION getCapacity() RETURNS INTEGER
    RETURN capacity
END FUNCTION



FUNCTION addRecord(element AS T)
    IF (size = capacity) THEN
        CALL resizeCapacity()
    END IF

    data[size] <- element
    size <- size + 1
END FUNCTION



FUNCTION resizeCapacity()
    DECLARE newCapacity AS INTEGER <- capacity * 2
    DECLARE newData AS NEW ARRAY OF T WITH newCapacity

    FOR( i FROM 0 TO size - 1 ) DO
        newData[i] <- data[i]
```

```
    END FOR

    DELETE data
    data <- newData
    capacity <- newCapacity
  END FUNCTION



      FUNCTION operator[index AS INTEGER] RETURNS T
    RETURN data[index]
  END FUNCTION


END CLASS
```

Test plan

Test id	Class	Test description	input	Expected output	Actual output	Result

1	date	Test a default constructor to check default values are assigned	date d;
			—
			date::date
			()
			{
			day=0;
			
			month=0;
			year=0;
			}
				0,0,0	0,0,0	pass

2	date	Test a constructor with parameters	date d(15, 5, 2026);	15,5,2026	15,5,2026	pass

3	date	Test setters	date d;
			
			d.setDay(31);
			
			d.setMonth(12);
			
			d.setYear(2022);
				31,12,2022	31,12,2022	pass

4	time	Default constructor	Time t;
			—
			Time::Time()
			{
			
			minute=0;
			hour=0;
			}
				00:00	00:00	pass

5	time	constructor with parameters	Time t(11,16);	16:11	16:11	pass
6	time	setters	Time t; t.setMinute(31);	12:31	12:31	pass

			t.setHour(12);			

7	Vector	Default constructor	Vector<int> v	size=0
				capacity=20
				—
				template<class T>
				Vector<T>::Vector()
				{
				size=0;
				
				capacity=20;
				data=new T[capacity];
				}
					size=0
					capacity=20
						pass

8	Vector	Template flexibility	Vector<string> vs;
			
			vs.addRecord("hello");
			
			vs.addRecord("world");
				vs[0] = “hello”
				vs[1]= “world;
					vs[0] = “hello”
					vs[1]= “world;
						pass

9	Vector	Deep copy using copy constructor	Vector<int> v1;
			
			v1.addRecord(100);
			
			Vector<int> v2(v1);
			
			v1[0] = 500;
				v1[0]= 500
				v2[0]=100
					v1[0]= 500
					v2[0]=100
						pass

10	Vector	Assignment	Vector<int	v2.size=1 v2[0]=10	v2.size=1 v2[0]=10	pass

		operator	> v1;
			
			v1.addRecord(10);
			
			Vector<int > v2;
			
			v2.addRecord(20);
			v2 = v1;			

11	Vector	Resize method	Vector<int > v;
			for(int i = 0; i < 21; i++) {
			
			v.addRecord(i);
			}
				21 elements added, Capacity is updated to 40	capacity= 40	pass

12	dataProcessor	Reading csv files inside a text file	// infile= text file stream
			
			while (getline(infile, csvName)) {
			if (!csvName.empty()) {
			
			cout << " found CSV file: " << csvName << endl;
			
			count++;
			}
			}
				found CSV file: MetData_Mar01-2014-Mar01-2015-ALL(in).csv
				
				found CSV file: MetData-31-3(in).csv
				
				found CSV file: MetData_Mar01-2015-Mar01-2016-ALL(in).csv
					found CSV file: MetData_Mar01-2014-Mar01-2015-ALL(in).csv
					
					found CSV file: MetData-31-3(in).csv
					
					found CSV file: MetData_Mar01-2015-Mar01-2016-ALL(in).csv
						pass

13	dataProcessor	Header parsing using string stream	// file stored in a csv file: T,SR,WAST,S
			
			ifstream infile(tempHeaderFile);
			
			if(getline(infile, headers)){
			
			stringstream ss(headers);
			
			while(getline(ss,column,',')){
			if (column == "WAST"){
			
			WAST = index;
			}
			if (column == "S"){
			S = index;
			}
			if (column == "T"){
			T = index;
			}
			if (column == "SR"){
			SR = index;
			}
				WAST=2, S=3, T=0, SR=1	WAST=2, S=3, T=0, SR=1	pass

			index++; } }			
14	**dataProcessor**	**Storing one line of data into Records**	**WAST,S,T,SR 1/03/2015 9:40,10.5,22.3,500**	**wind Speed: 10.5 temp: 22.3**	**wind Speed: 10.5 temp: 22.3**	**pass**
15	**compute**	**Calculates average wind speed in km/s**	**10.0f And 20.0f //these numbers are based on m/s, 10* 3.6 =36.0km/h**	**54.0**	**54.0**	**pass**
16	**compute**	**Calculates average temperature**	**20.0f and 30.0f**	**25.0**	**25.0**	**pass**
17	**compute**	**Calculates total solar radiation in kwh**	**600.0f and 1200.0f // formula= (n/6.0)/1000.0**	**0.3**	**0.3**	**pass**
18	**compute**	**Calculate standard deviation**	**10.0f and 20.0f**	**5.0**	**5.0**	**pass**
19						

While initial testing focused on the Unit level to ensure memory safety and algorithmic correctness (such as Vector resizing and Deep Copy logic), the plan naturally evolved into Integration Testing. This was necessary to validate the interfacing between the software and the host file system—specifically ensuring the dataProcessor could navigate the directory structure.

Output of test run

```
   weatherCSVReader test program
-----------------------------------------

1. tesing DATE class
test one: default constructor
expected:0/0/0
actual:0/0/0

test two: constructor with parameters
expected:15/5/2026
actual:15/5/2026

test three: setters
expected:31/12/2022
result: pass!!

2. tesing TIME class
test one: default constructor
expected:0/0
actual:0/0

test two: constructor with parameters
expected:16:11
actual:16:11

test three: setters
expected:12:31
result: pass!!

3. testing VECTOR template class
```

```
test one: default constructor and initial state
initial size: 0 (expected: 0)
initial capacity: 20 (expected: 20)

test two: template flexibility
stored: hello world
result: pass

test three: copy constructor
value of v1[0]: 500
 value of v2[0]: 100
result: pass (successful deep copy)

test four: assignment operator
v2 size after assignment operator: 1 (expected: 1)
value of v2[0]: 10 (expected: 10)

test five: resizing check
size after 21 elements adds: 21
new capacity: 40 (expected: 40)

4. tesing DataProcessor class

test: reading CSV list
  found CSV file: MetData_Mar01-2014-Mar01-2015-ALL(in).csv
```

```
  found CSV file: MetData-31-3(in).csv
  found CSV file: MetData_Mar01-2015-Mar01-2016-ALL(in).csv
result: Pass

test: header parsing
expected indexes: WAST=2, S=3, T=0, SR=1
 actual: WAST=2, S=3, T=0, SR=1

test: storing one line into Records
  record stored successfully.
  wind Speed: 10.5 (expected 10.5)
  temp: 22.3 (expected 22.3)
5. testing COMPUTE class

test one: average wind speed calculation
expected: 54.0, actual: 54
result: pass

test two: average temperature
expected: 25.0, actual: 25
result: pass

test three: total solar radiation (kWh Conversion)
expected: 0.3, actual: 0.3
result: pass

test four: standard deviation
expected: 5.0, actual: 5
result: pass
```